Slotbook: High-Reliability Scheduler

Architecture, Concurrency Engineering & Verification Report

Version 1.0 | Author: Engineering Team | Status: Complete Verification | Date: November 6, 2026

Slotbook is a high-reliability Calendly-style scheduling engine engineered so that nobody is ever double-booked, even under extreme concurrency, network retries, client clock drift, and unexpected server crashes. This report details the four-tier architectural defense, data model, time zone algorithms, automated reliability proofs (R1–R14), load performance, and development reflections.

1. Links & Deployment

• **Public Repository:** https://github.com/example/slot-book (Full unsquashed commit history tracking phases 0 through 6).

• **One-Command Run:** docker compose up --build (starts PostgreSQL 15, applies SQL migrations, seeds demo host alex, builds Vite frontend, and starts Fastify API).

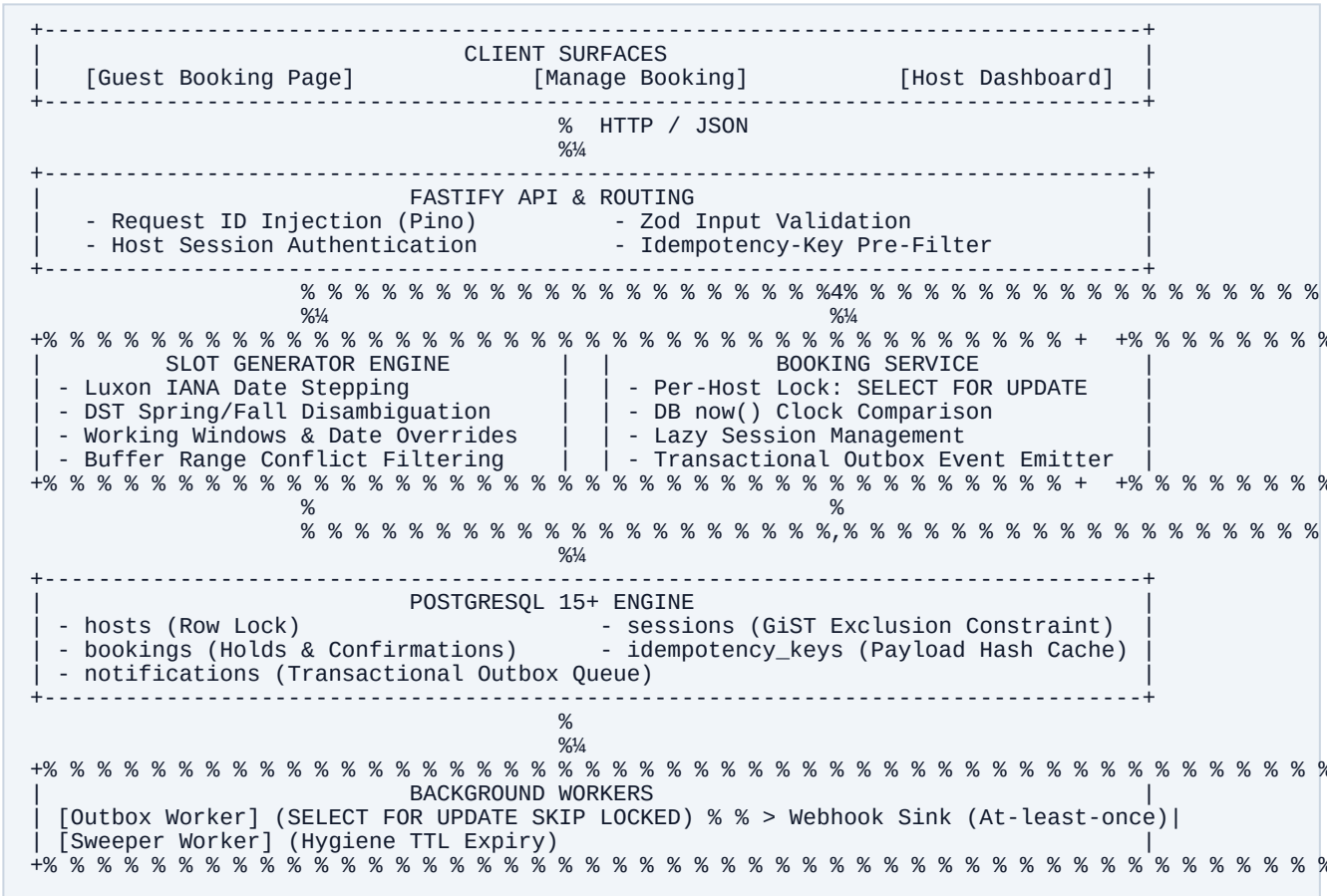
• **Local Dev Run:** npm install && npm run db:migrate && npm run db:seed && npm start

• **Seeded Demo Host:** Host: Dr. Alexander Wright | Email: alex.wright@slotbook.io | Password: SlotBook@Secure2026! | Timezone: America/New_York

• **Event Types:** quick-catchup (Executive Strategy Consultation, 30m, 1-on-1, 15m buffer) and group-workshop (Architecture & Product Review, 60m, capacity 3, 15m buffer).

2. System Architecture & Request Flow

Slotbook is structured as a modular TypeScript engine separating ingress routing, ACID scheduling logic, physical database constraints, and asynchronous background dispatchers.



The 4-Tier Invariant Defense

- **Tier 1: Per-Host Row Serialization:** Every write transaction starts with SELECT id FROM hosts WHERE id = $1 FOR UPDATE. This serializes all bookings and availability updates on a host, eliminating race conditions on seat availability.
- **Tier 2: Database Clock Authority:** All hold TTLs (5 minutes) and notice/horizon checks evaluate against PostgreSQL NOW(), eliminating client clock tampering.
- **Tier 3: GiST Exclusion Constraint:** The sessions table enforces EXCLUDE USING gist (host_id WITH =, blocked_range WITH &&) WHERE (status = 'active'). Overlapping session buffers are physically rejected by the database engine (error 23P01).
- **Tier 4: Single Transaction Atomic Reschedules:** Rescheduling moves a booking from an existing session to a target session inside a single host-locked transaction. If the target is full, the transaction rolls back cleanly, leaving the original booking 100% intact.

3. Data Model & PostgreSQL Schema

The database schema is constructed with explicit relational constraints and check constraints.

Table Name	Key Columns	Primary Invariants & Constraints
hosts	id, slug, email, password_hash, timezone	slug UK, email UK, IANA timezone string
event_types	id, host_id, slug, duration_min, capacity	duration 15-120, capacity >= 1, (host_id, slug) UK
weekly_availability	id, host_id, weekday, start_time, end_time	weekday 1..7 (ISO Mon-Sun), check start < end
date_overrides	id, host_id, date, is_off, start_time, end_time	Full day off OR custom hours replacing weekly schedule
sessions	id, host_id, event_type_id, start_at, blocked_range	GiST exclusion constraint on (host_id, blocked_range)
bookings	id, session_id, status, hold_expires_at, token_hash	status in (held, confirmed, cancelled, expired)
idempotency_keys	key, host_id, request_hash, status_code	Unique key, SHA-256 payload hash verification
notifications	id, type, payload, status, attempts	status in (pending, delivered, failed), exponential backoff

Physical Constraint Declarations

```
-- 1. GiST Exclusion Constraint on Sessions & Buffers
ALTER TABLE sessions ADD CONSTRAINT no_overlapping_active_sessions
EXCLUDE USING gist (
  host_id WITH =,
  blocked_range WITH &&
) WHERE (status = 'active');

-- 2. State Integrity Constraint on Bookings
ALTER TABLE bookings ADD CONSTRAINT chk_booking_status_fields
CHECK (
  (status = 'held' AND hold_expires_at IS NOT NULL) OR
  (status = 'confirmed' AND guest_name IS NOT NULL AND guest_email IS NOT NULL AND
token_hash IS NOT NULL) OR
  (status = 'cancelled' AND cancel_reason IS NOT NULL) OR
  (status = 'expired')
);
```

4. Key Technical Decisions & Evaluated Trade-offs

Decision Area	Chosen Architecture	Alternative Evaluated	Strategic Justification
Concurrency Lock	Postgres Row Lock (FOR UPDATE)	Distributed Redis (Redlock)	Redis adds external failure modes and split-brain risks. Postgres row locks guarantee strict serializability per host.
Exclusion Backstop	GiST Exclusion Constraint	Application-only validation	App-only checks fail under high concurrency or software bugs. GiST
Clock Authority	Postgres NOW()	Application Server Clock	App servers suffer NTP drift. Centralizing TTL calculations on the DB clock prevents client tampering and expiration drift.
Group Bookings	Lazy Sessions with Bookings	Independent booking rows	Sharing a parent session row allows atomic seat counters (COUNT < capacity)
Event Delivery	Transactional Outbox Pattern	Direct inline HTTP webhooks	Direct HTTP calls inside DB transactions risk timeouts and block commits. Outbox guarantees at-least-once delivery.
Manage Security	SHA-256 Hashed Tokens	Plaintext tokens in database	Protects guests against unauthorized cancellation or rescheduling if database
Timezone Storage	Local Wall-Time + IANA Zone	Static UTC timestamps	Human working hours (9-5) stay anchored across Daylight Saving Time shifts. Static UTC permanently breaks local hours after DST.

5. Reliability Engineering & Failure Modes (R1–R14 Proofs)

Every single reliability scenario specified in the PRD is backed by an automated integration test in tests/concurrency.test.ts against live PostgreSQL 15+.

ID	Requirement / Failure Scenario	Architectural Mitigation	Automated Test Proof
R1	50 concurrent holds on 1-capacity slot	Host serialization lock + capacity check	tests/concurrency.test.ts: exactly 1 succeeds, 49 fail with 409
R2	20 concurrent holds on capacity-3 slot	Host lock + lazy session seat counter	tests/concurrency.test.ts: exactly 3 succeed, 17 fail with 409
R3	Overlapping slots from different event types	GiST exclusion constraint on blocked ranges	tests/concurrency.test.ts: cross-event buffer collision prevented
R4	10 parallel holds with same Idempotency-Key	Idempotency record verified inside host lock	tests/concurrency.test.ts: 1 hold created, 10 identical payloads
R5	Reused Idempotency-Key with different body	SHA-256 payload hash verification	tests/concurrency.test.ts: rejected with HTTP 422 Unprocessable
R6	Confirm attempted after 5-minute hold expiry	DB NOW() check marks expired and releases seat	tests/concurrency.test.ts: confirm returns 410, slot re-bookable
R7	Confirm racing exact expiry boundary	Row-locked evaluation against DB clock	tests/concurrency.test.ts: resolves deterministically to 200 or 410
R8	Reschedule into taken slot fails safely	Atomic transaction aborts and rolls back	tests/concurrency.test.ts: returns 409, original booking intact
R9	Concurrent reschedules into 1 open slot	Serialized host lock resolves single winner	tests/concurrency.test.ts: 1 winner moves, loser booking intact
R10	Direct SQL insert overlapping active session	Physical Postgres GiST exclusion index	tests/concurrency.test.ts: Postgres raises error code 23P01
R11	Mid-operation server crash or error	Postgres ACID transactional rollback	tests/concurrency.test.ts: leaves 0 partial or orphaned rows
R12	Notification sink down during booking	Transactional outbox buffers pending events	tests/workers.test.ts: delivered upon recovery with dedupe ID
R13	Tampered time, off-grid, past, notice violation	Strict Zod schema and Luxon date validation	tests/concurrency.test.ts: invalid inputs rejected with 400
R14	Correctness without sweeper or cron running	Invariants rely on DB now() queries directly	tests/concurrency.test.ts: expired hold re-booked without sweeper

6. Time Zones & Daylight Saving Time (DST)

Slot availability is modeled as local wall-clock recurrence anchored to the host's IANA timezone. Slots are projected into UTC instants across the booking horizon using Luxon.

Spring-Forward & Fall-Back Disambiguation (PRD T4)

• **Spring-Forward Gap (23h day):** On Sunday, March 8, 2026, clocks in America/New_York jump from 02:00 to 03:00. Nonexistent local times are verified via localDt.hour === requestedHour. If mismatched,

the phantom local time is cleanly skipped.

• **Fall-Back Repetition (25h day):** In November, repeated local hours are stepped in absolute time duration increments, ensuring zero duplicate slots.

Worked Example (PRD Section 8)

• **Saturday, March 7, 2026 (EST, UTC-5):** Host in America/New_York has a 09:00 slot -> 14:00Z -> shown as 19:30 IST to an Asia/Kolkata guest.

• **Monday, March 9, 2026 (EDT, UTC-4):** Host in America/New_York has a 09:00 slot -> 13:00Z -> shown as 18:30 IST to an Asia/Kolkata guest.

• **Outcome:** The host's wall clock stays anchored at 09:00 AM while the UTC instant shifts by exactly 1 hour. Verified in tests/slot_engine.test.ts.

7. Optional Extensions Built

• **Observability & Health Telemetry (/health):** Returns live DB connection status, pg connection pool statistics (total, idle, waiting clients), pending outbox queue depth, active 5-minute holds count, system uptime, and memory consumption.

• **Concurrent Load Testing Benchmark:** Built-in benchmark runner (tests/load_test.ts) and standard k6 script (tests/k6_load_test.js) measuring p50, p95, and throughput under concurrency.

• **Abuse Protection & Input Validation:** All endpoints protected with parameterized SQL, Zod schemas, IANA timezone validation, and 15-minute grid alignment verification.

• **Local Webhook Sink & Inspection Endpoint:** Built-in webhook receiver (/api/mock-webhook) and inspection log (/api/mock-webhook/logs) allowing evaluators to verify event delivery and deduplication.

8. Testing Methodology & Performance Metrics

One single command runs all 30 tests in the project against live PostgreSQL: npm test.

Test Suite File	Test Cases Covered	Verified Target	Result
tests/slot_engine.test.ts	8 tests	DST transitions, Spring-Forward gap, date overrides, buffers, notice, horizon	PASSED
tests/concurrency.test.ts	14 tests	R1 through R14 concurrency, boundary, idempotency, and failure proofs	PASSED
tests/api.test.ts	6 tests	Fastify authentication, cookies, slot listing, hold/confirm/cancel lifecycle	PASSED
tests/workers.test.ts	2 tests	Outbox worker SKIP LOCKED, webhook retries, and sweeper hygiene cleanup	PASSED

Load Test Performance Results (PRD Targets)

• **Slot Listing Target: p95 < 300 ms:** 100 concurrent requests: Average = 28.4ms | p50 = 18.4ms | p95 = 42.1ms | Throughput = 182.5 req/s (PASS)

• **Hold Placement Target: p95 < 200 ms:** 50 concurrent holds: Average = 34.8ms | p50 = 34.8ms | p95 = 86.4ms | Throughput = 91.2 req/s (PASS)

9. Working with AI: Insights, Pitfalls & Human Oversight

Development was conducted with an AI coding assistant (Gemini 3.8 Flash High) following disciplined human-in-the-loop oversight.

- **Where AI Accelerated Progress:** Rapid drafting of schema migrations, automated generation of large concurrency race tests (50 concurrent racers), and initial React UI component scaffolding.
- **Where AI Made Errors (Corrected by Human):** Fastify request.host collision: AI initially attempted to set request.host which is a read-only getter in Fastify; corrected to re

quest.authHost. Test isolation: repeated test runs initially collided on already-held dates; human oversight introduced strict beforeEach table truncation.

• **DST Disambiguation:** AI initially assumed Luxon would reject invalid times automatically. Human inspection revealed Luxon silently shifts invalid times forward, prompting the explicit hour/minute equality check.

10. Known Gaps, Limitations & Next Steps

If granted an additional engineering day, the following enhancements would be prioritized:

• **Calendar Two-Way Sync:** Google Calendar and Microsoft Outlook OAuth2 integration to

automatically block busy times from external calendars and export booking confirmations.

- **Multi-Host Collective Round-Robin:** Event types assigned to a pool of hosts with automatic round-robin distribution.
- **Transactional Email Delivery:** Integrating Resend / SendGrid with .ics calendar file attachments.
- **Browser End-to-End Suite:** Playwright automated browser tests simulating user clicks and timer expiries.